

Week 12— Deadlock and Prevention

This program demonstrates a deadlock scenario where two threads acquire two resources in different orders.

```
GNU nano 8.7.1 week12b.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 1 locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}
```

```
sri_dhruthi@Dhruthi:~$ nano week12b.c
sri_dhruthi@Dhruthi:~$ gcc week12b.c -o week12b -pthread
sri_dhruthi@Dhruthi:~$ ./week12b
Thread 2 locked Resource 1
Thread 2 locked Resource 2
Thread 1 locked Resource 1
Thread 1 locked Resource 2
Program completed without deadlock.
```